

Workshop · Half day · 180 minutes

RAG that survives your actual documents

You leave with twenty questions your own index cannot answer yet, and the named reason each one failed.

Varun Tyagi & Renuka Adnani · Raygency

September 12, 2026 · raygency.com

How this runs

0:00	Why the demo worked Twelve clean documents, and then the real folder arrived
0:40	Filing, not searching You audit your own corpus and find what it cannot answer
1:35	Measuring the retrieval, not the answer Twenty questions, and the document that should win each one
2:25	Grounding and the bill Citations, refusals, and what the whole thing costs a month

Four true things. One wrong answer.

1

The document was there

Indexed on Monday.
Nothing failed, nothing timed out.

2

The model was the good one

The same model that nailed every question in the demo.

3

The prompt was careful

Written by the person who knows the policy best.

4

The answer was confident

Fluent, structured, with a citation at the end of it.

It cited the 2019 Christmas party memo.

Nothing broke. The right document was in the corpus and was never retrieved.

Your retrieval problem is a filing problem

By the end of this block you can name which of six failure modes your own system has, and what each one costs you.

Every newsroom had a clippings desk

1

Somebody filed it

Every story was cut out, dated, and put in a labelled folder the same afternoon.

2

Somebody pulled it

A reporter said a name and got back a thin stack, not the whole cabinet.

3

Somebody printed it

With a byline and a dateline, so a reader could go and check.

Nobody ever fixed a bad clippings desk by hiring a better reporter.

Retrieval is three jobs wearing one name

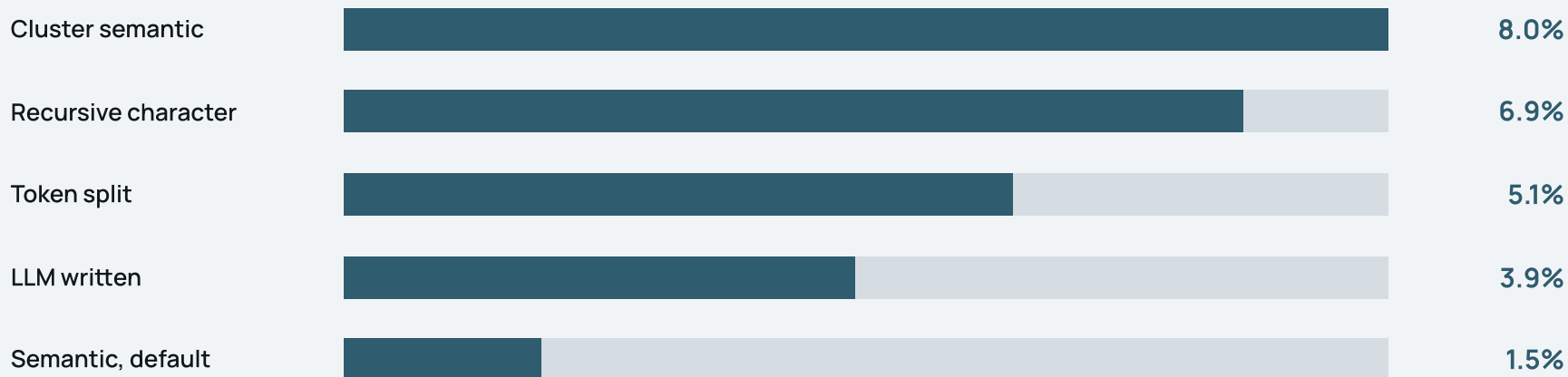
Index	Cut it up and file it Split each document into chunks, turn each chunk into a vector, store it with whatever you kept about where it came from
Search	Pull the thin stack Turn the question into a vector, return the k nearest chunks. Usually k is 5. Nobody in the room ever chose 5
Write	Answer from the stack Hand those chunks to the model with the question. It answers from what it was handed, and from nothing else

The model never sees your corpus. It sees five paragraphs somebody else picked.

Six ways it breaks, and who pays

FAILURE	WHAT YOU SEE	WHAT IT COSTS
Not indexed	Confident answer, wrong source	Trust, once
Chunked mid sentence	Half a rule, stated as whole	A wrong decision
Similar, not relevant	Five chunks on the theme	Rework, every query
Stale index	Last quarter's policy	The compliance call
No provenance	Nobody can verify it	Adoption stalls
Scanned page, no text	Silence on the key document	The whole use case

Most of what it retrieves is not the answer



Recall across all seven chunkers moved eight points. How much of the window was worth reading moved five fold.

Chroma, Evaluating Chunking Strategies for Retrieval, July 2024. Share of retrieved tokens that were the answer, at $k = 5$, text-embedding-3-large, five corpora.

Same question. Same index. Two answers.

ASKED

How many days of parental leave do we offer in Germany?

ANSWERED

Sixteen days, per the attached policy. The retrieved chunk was an internal events memo that used the phrase "family days" twice.

It did not hallucinate. It answered accurately from the wrong paragraph.

Every eval you own is pointed at the answer. Nothing you own is pointed at the paragraph.

Similar and relevant are not the same word

WHAT THE VECTOR KNOWS

WHAT THE VECTOR CANNOT KNOW

These two paragraphs
use the same
vocabulary

Which one is current

They sit near each other
in meaning space

Which one applies to Germany

One scores 0.81, the
other 0.79

Which is the policy and which summarises it

Everything in the right hand column is metadata you either kept or threw away at indexing time.

Nobody opens the folder before they index it

1

Count the duplicates

Four near identical versions of one policy will split your retrieval four ways.

2

Count the undated

A document with no date in it cannot lose to a newer one. It will win forever.

3

Sort by bytes per page

A scanned page is huge and returns no text at all. That one sort finds every silent file in thirty seconds, and no ingest log will tell you.

One hour with the folder open beats one week of tuning.

You will find the reason for three of your six failure modes before you write any code.

A chunk is a paragraph with an address

By the end of this block you have audited your own corpus and can say which documents it will never answer from.

A chunk has to stand up on its own

1

It says what it is about

"As stated above, this does not apply" retrieves perfectly and means nothing.

2

It answers one thing

If it spans two rules, it will be retrieved for both and be half right for each.

3

Split on the headings

Before anything clever. Your policies already number their own sections, so one afternoon of regex buys structure aware chunking.

If a new joiner could not answer from the chunk alone, neither can the model.

Read three chunks out loud at random. That is the whole test, and it takes four minutes.

Split on structure, not on character count

APPROACH	WHAT IT COSTS	WHAT IT BUYS
Fixed 800 characters	Two lines of code	Cuts tables in half
Add 100 of overlap	Nothing	Hides it, does not fix it
Split on headings	One afternoon	Rule keeps its exception
Parser per doc type	Two days, once	Heading path, free

Structure aware chunking is the single change with the largest effect per hour spent.

Shaukat, Adnan and Kuhn, arXiv 2603.06976, March 2026. Paragraph group chunking scored 24% Precision@1 against 2 to 3% for fixed size character splitting, across 36 methods and six domains.

Five fields you wish you had kept

FIELD	WHAT IT BUYS YOU	COST TO ADD
Effective date	Newest version wins	One regex
Jurisdiction	Filter before you search	Folder path
Document type	Policy beats summary	Filename rule
Heading path	The chunk knows its topic	Free, if parsed
Source URL	The reader can verify	Free, at ingest

Bruni, Avvenuti, Tonellotto and Tesconi, AMAQA, arXiv 2505.13557, May 2025. Holding these as real fields rather than as text inside the chunk moved answer accuracy from 0.50 to 0.86 on GPT-4o.

Nobody gets stuck at the door

A · RUN IT

Your own corpus

You have a folder and a terminal.
Point the script at your real documents.

B · LITE

Any chat window

Paste five of your real documents in
and run the same audit by hand.
Same lesson, smaller n.

C · WATCH

Follow on screen

Use the sample corpus we hand over
and run it at your desk tonight.

CHECKPOINT

Call out your letter. A, B or C.

Audit the folder before you trust the index

TYPE THIS

```
For each file: name, page count, is  
any text extractable, newest date  
inside, and does another file repeat  
its first paragraph.
```

One rule: point it at the real folder, not a cleaned copy. Ask for one row per file and no summary. Notice the rows with an empty text column and the rows with no date.

CHECKPOINT

How many of your files came back with no extractable text? Say the number out loud.

You cannot fix what you have never scored

By the end of this block you have twenty questions, the document that should win each one, and a number you can watch move.

Recall at k in one sentence

1

Write the question

A real one, from a real person, phrased the way they actually phrase it.

2

Name the document

The one a knowledgeable colleague would hand over. One line, by hand, before you run anything.

3

Score at 5 and at 10

In the same run. In the top ten but not the top five is a re-ranking problem and an afternoon. In neither is an indexing problem and a week. Two numbers, and they tell you which.

It takes ninety minutes to build and it is the only number in this deck that predicts anything.

You are not debugging an answer. You are debugging a retrieval.

Every hour spent rewriting the prompt to fix a wrong fact is an hour spent on the wrong layer.
Score the retrieval first. Then the answer is a separate, smaller problem.

Twenty rows tell you more than a dashboard

QUESTION	SHOULD RETRIEVE	TOP 5
Parental leave in Germany	HR policy 2026, sec 4	No
Notice period, 3 year staff	Contract template v7	Yes
Who signs a 40k invoice	Approval matrix 2026	Yes
Data retention, CVs	Retention schedule 2025	No

If your set has no failures in it, you wrote the set to pass.

Score the stack before you score the story

	RETRIEVAL EVAL	ANSWER EVAL
What it asks	Did the right one come back	Was the answer right
Who decides	A string match	A human or a grader
What it costs	Seconds, nothing	Minutes, real money
When to run it	First, always	After retrieval is fixed

Run them together and every failure looks like a model problem.

Build the set and get your first number

TYPE THIS

```
Run these twenty questions. Return  
the top five chunk sources for each,  
and mark whether the expected  
document is among them.
```

One rule: do not change the questions after you see the score. Report the count, then only the failures, and notice whether they share a document type or a date range.

CHECKPOINT

What is your recall at 5, out of 20? First number, not the tuned one.

An answer nobody can check is not an answer

By the end of this block you can cite, refuse, and say what the whole thing costs a month.

The byline and the dateline do the trust work

1

Cite the chunk, not the file

"HR policy 2026, section 4" sends them to the paragraph. A filename sends them on a search.

2

Show the date

A reader who sees 2019 next to the answer catches the error you never will.

3

Print the chunk, not just the link

During the pilot, show the retrieved paragraph beside the answer. Your reviewers will find retrieval bugs you would never have written a test for, while reading for something else.

Citations are not decoration. They are how the room finds your retrieval bugs for free.

Refusing is a product feature, not a failure

1

Set a floor

Below a similarity score you pick, return nothing and say so. Pick it from your own eval rows.

2

Say what is missing

"No current policy covers this in Germany" is useful. "I cannot help with that" is not.

3

Route the refusal

Every refusal is a question your corpus should answer and does not. Log them and read them weekly.

A system that never refuses is a system that always answers, including when it should not.

Your refusal log is the best backlog you will ever get, and it writes itself.

Five levers, and what each one moves

LEVER	EFFECT ON COST	EFFECT ON QUALITY
Smaller k	Down, linearly	Down, if recall is weak
Re-rank top 20 to 5	Up a little	Up a lot
Cache frequent questions	Down hard	Flat, if invalidated
Cheaper embedding model	Down at ingest only	Down, measurably
Cheaper answer model	Down hard	Flat, once grounded

Put your own four inputs in before you quote any of this to somebody who signs things. Every cost figure we hand you is a model, not a case study.

Every model on this slide gets replaced within a year.

The folder audit, the twenty questions and the five fields you kept do not. That is the whole reason we spent an afternoon on filing instead of on prompts.

Three things I still get wrong

01

Tables inside PDFs

I have no reliable general answer. I parse them per document type, and it does not transfer to the next client.

02

The research disagrees with me

NAACL 2025 tested 25 configurations and found fixed size chunking held up fine. A March 2026 study across 36 methods found structure aware winning about ten to one. I teach the second and I could be wrong.

03

Questions that span ten documents

"Summarise every exception across all policies" is not a retrieval question, and no value of k makes it one. Decompose it or route it away.

Qu, Tu and Bao, Findings of NAACL 2025, arXiv 2410.13070, against Shaukat, Adnan and Kuhn, arXiv 2603.06976, March 2026. Both are on the reading list we hand over.

Two things you keep, and one piece of homework

1

Keep the audit

The folder report from Round 1. Re-run it every time the corpus changes and diff the two.

2

Keep the twenty

Your question set. It is the only thing here that still works after the model you use gets replaced.

3

The homework

One set, one Friday. Run it and send me the first row where the right document was in the corpus and retrieval missed it.

Nobody is going to tell you the answer was wrong.

They will quietly stop using it, and the usage chart will look like a bad launch rather than a retrieval bug. Your twenty questions are the only alarm you get.